The Linux Completely Fair Scheduler (CFS)

Sistema Operativos II

LCC - FCEIA-UNR

14 de abril de 2026

The Linux Completely Fair Scheduler (CFS)

- El planificador *Completely Fair Scheduler* (CFS) implementa una política de planificación por reparto equitativo (*fair-share*).
- Implementado por Ingo Molnar e integrado en Linux 2.6.23 (2007).
- Su objetivo es dividir de manera equitativa el uso de la CPU entre todos los procesos en competencia.
- Esto se logra mediante una técnica simple basada en conteo, conocida como virtual runtime (`vruntime`).
- Cuando ocurre una decisión de planificación, CFS selecciona para ejecutar a continuación el proceso con menor virtual runtime .
- Esto plantea una pregunta: ¿cómo determina el planificador cuándo detener el proceso en ejecución y dar lugar al siguiente?

The Linux Completely Fair Scheduler (CFS)

- Existe un compromiso en CFS: aumentar la frecuencia de cambios de contexto mejora la equidad pero degrada el rendimiento, mientras que reducirla mejora el rendimiento a costa de una menor equidad en el corto plazo.
- CFS gestiona este compromiso mediante diversos parámetros de control.
- `sched_latency` es el tiempo total en el que todos los procesos listos deberían ejecutarse al menos una vez.
- Un valor típico de `sched_latency` es 48 milisegundos.
- CFS divide este valor por el número de procesos (n) en ejecución para determinar el *time slice* de cada uno, asegurando así que, en ese intervalo, la asignación de CPU sea completamente equitativa.

The Linux Completely Fair Scheduler (CFS)

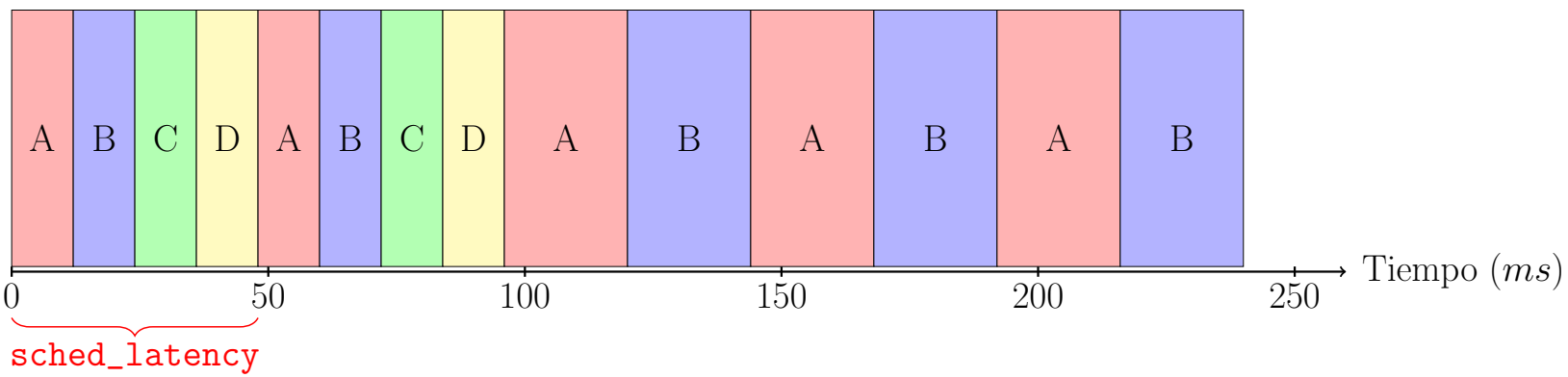
- Por ejemplo, si hay $n = 4$ procesos en ejecución, CFS divide el valor de `sched_latency` por n, obteniendo un *time slice* de 12 *ms* por proceso.
- CFS planifica el primer proceso y lo ejecuta hasta que ha consumido 12 *ms* de `virtual runtime`.
- Luego, verifica si existe otro proceso con menor *vruntime* para ejecutar.
- Si existe, CFS cambia a uno de los otros tres procesos, y así sucesivamente.

The Linux Completely Fair Scheduler (CFS)

- ¿Qué ocurre si hay "demasiados" procesos en ejecución? Esto podría implicar *time slices* muy pequeños y, en consecuencia, demasiados cambios de contexto.
- Para abordar este problema, CFS introduce otro parámetro: `min_granularity`, típicamente de alrededor de 6 *ms*.
- CFS nunca asigna a un proceso un *time slice* menor que este valor, evitando así un exceso de sobrecarga por planificación.
- Por ejemplo, si hay diez procesos en ejecución, el cálculo original dividiría `sched_latency` por diez, dando un *time slice* de 4.8 *ms*.
- Sin embargo, debido a `min_granularity`, CFS asigna un *time slice* de 6 *ms* a cada proceso.
- Aunque en este caso CFS no logra una equidad perfecta dentro del intervalo objetivo (`sched_latency` de 48 *ms*), se mantiene cercano a ella, al mismo tiempo que preserva una alta eficiencia de CPU.

The Linux Completely Fair Scheduler (CFS)

- La figura muestra un ejemplo en el que cuatro procesos (A, B, C, D) se ejecutan durante dos *time slices* de 12 ms cada uno.
- Luego, dos de ellos (C, D) finalizan, quedando solo dos procesos, que pasan a ejecutarse durante 24 *ms* cada uno en un esquema tipo round-robin.



The Linux Completely Fair Scheduler (CFS)

- CFS utiliza una interrupción de temporizador periódica, lo que implica que solo puede tomar decisiones en intervalos de tiempo fijos.
- Esta interrupción ocurre con frecuencia (por ejemplo, cada $1\,ms$), brindando a CFS la oportunidad de activarse y determinar si el proceso actual ha alcanzado el final de su ejecución.
- Si un proceso tiene un *time slice* que no es un múltiplo exacto del intervalo de la interrupción del temporizador, no hay problema.
- CFS lleva un seguimiento preciso del `vruntime`, lo que implica que, a largo plazo, terminará aproximando un reparto ideal de la CPU.

The Linux Completely Fair Scheduler (CFS)

- CFS también permite controlar la prioridad de los procesos, posibilitando que usuaries o administradores otorguen a ciertos procesos una mayor porción de CPU.
- Esto no se realiza mediante tickets, sino a través de un mecanismo clásico de UNIX conocido como el nivel de `nice` de un proceso.
- El parámetro `nice` puede tomar valores entre -20 y +19, con un valor por defecto de 0.
- Valores positivos de `nice` implican menor prioridad, mientras que valores negativos implican mayor prioridad.
- Cuando un proceso es "demasiado amable" (`nice`), simplemente recibe menos atención del planificador.

The Linux Completely Fair Scheduler (CFS)

- El *time slice* de cada proceso es variable y se calcula aproximadamente como:

$$\text{time slice}_k = \frac{\text{weight}_k}{\sum_{i=0}^{n-1} \text{weight}_i} \cdot \text{sched latency}$$

  donde el término $weight_k$ representa el peso (peso relativo de prioridad) del proceso k.

- Estos pesos permiten calcular el *time slice* efectivo de cada proceso (como antes), pero ahora teniendo en cuenta sus diferencias de prioridad:
  - A mayor peso $\rightarrow$ más CPU
  - A menor peso $\rightarrow$ menos CPU
- La relación entre prioridad y peso no es lineal, sino exponencial/logarítmica. Esto permite cambios suaves y consistentes en la prioridad, evitando saltos bruscos en el reparto de CPU.

The Linux Completely Fair Scheduler (CFS)

- Además de generalizar el cálculo del *time slice*, la forma en que CFS calcula el `vruntime` también debe adaptarse.
- Esta es la nueva fórmula, que toma el tiempo real de ejecución que el proceso *i* ha acumulado ($runtime_i$) y lo escala de manera inversa según el peso del proceso:

$$\mathrm{vruntime}_i = \mathrm{vruntime}_i + \frac{\mathrm{weight}_0}{\mathrm{weight}_i} \cdot \mathrm{runtime}_i$$

donde weight_0 es un peso de referencia que corresponde al peso asociado al valor de `nice` igual a 0.

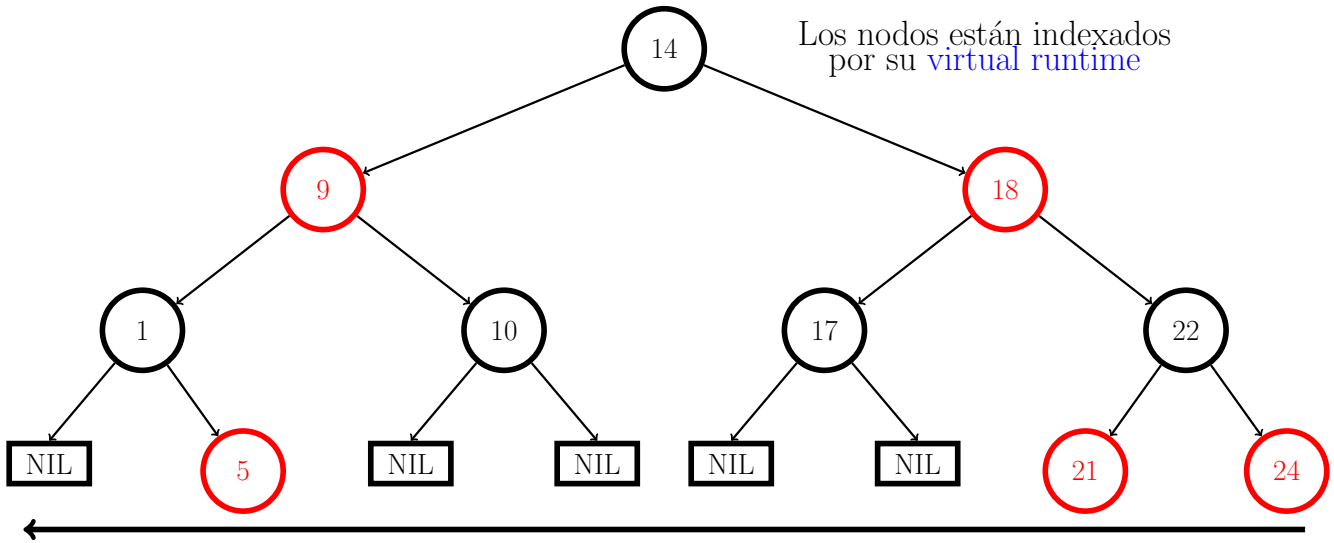
The Linux Completely Fair Scheduler (CFS)

- CFS selecciona el próximo proceso a ejecutar manteniendo los procesos *runnable* en un árbol rojo-negro.
- Un árbol rojo-negro es un árbol binario de búsqueda auto-balanceado que utiliza reglas de coloreo para mantener el balance.
- CFS ejecuta siempre el proceso con menor `vruntime` (es decir, el más atrasado en uso de CPU).

The Linux Completely Fair Scheduler (CFS)

Las reglas de un árbol rojo-negro permiten mantener una complejidad temporal logarítmica:

1. Cada nodo es rojo o negro.
2. La raíz es negra.
3. Todos los nodos NIL son negros.
4. No puede haber dos nodos rojos consecutivos.
5. Todo camino desde un nodo hasta un NIL tiene la misma cantidad de nodos negros (black-height).



Bibliografía



Arpaci-Dusseau, R. H. and Arpaci-Dusseau, A. C. (2018).

Operating Systems: Three Easy Pieces.

http://www.ostep.org.



Joshi, V. (Acceso: 13/04/2026).

Painting Nodes Black With Red-Black Trees.

https://medium.com/basecs/painting-nodes-black-with-red-black-trees-60eacb2be9a5.



The Linux Kernel documentation (Acceso: 13/04/2026).

CFS Scheduler.

https://docs.kernel.org/scheduler/sched-design-CFS.html.